

Prague University of Economics and Business

Faculty of Informatics and Statistics



BUILDING AGILE TEAM PRODUCTIVITY DASHBOARD

BACHELOR THESIS

Study programme: Applied informatics

Field of study: Applied informatics

Author: Iuliia Loseeva

Supervisor: doc. Ing. Ota Novotny, Ph. D

Prague, January 2022

Acknowledgement

I hereby wish to express my appreciation and gratitude to the supervisor of my thesis, doc. Ing. Ota Novotny, Ph. D and my family

Abstract

The topic of this thesis is a Jira Atlassian as a project productivity tool, which aim is to formalize agile technology and adapt it to the precise company requirements. The Jira allows users to create their projects and to track the progress in Agile or Kanban methodology. The work describes various tools, which could be use for this purpose and reason why the Jira could be selected as a main one in the company.

The purpose of this work was to create various charts using Jira, which could show a team or department productivity and help to improve day-to-day work in the company. The whole process of dashboard creation was described in the practical part. There are arguments why dashboard should be created in Jira. To detect which charts should be created the short survey among the team was provided. After the dashboard was created it was presented to the team and the feedback was collected of how they evaluate created dashboard and what benefits do they see.

In the theoretical part the principle of the agile methodology is described. The analysis of the productivity tracking software market is provided, and Jira tool is described in the details.

Keywords

jira, scrum, kanban, agile, productivity , dashboard.

Content

Introduction	6
Topic definition.....	6
Aim of the work.....	6
The target audience	6
Methods used.....	6
Work structure.....	7
Work outputs	7
1 Analysis of the Productivity Software needs of the Agile Team.....	8
1.1 Agile	8
1.2 Agile frameworks overview	9
1.3 Software Team Productivity Metrics.....	10
1.4 Productivity Charts.....	11
1.5 Risks of using the productivity charts	14
1.6 Conclusion	15
2 Analysis of the Productivity tracking software market	16
2.1 Productivity Tracking software requirements	16
2.2 Productivity Tracking software market competitors	17
2.3 Conclusion	18
3 JIRA: tool overview and capabilities for agile teams	21
3.1 JIRA functionality and intended usage.....	21
3.2 Productivity Reporting in JIRA	22
3.3 Conclusion	23
4 Practical Part: building productivity charts	24
4.1 Describing the use-case.....	24
4.2 Selecting the visualizations.....	25
4.3 Implementation description	27
4.3.1 Not analyzed tickets in production.....	27
4.3.2 Distribution of tickets between teams	28
4.3.3 Distribution of tickets depends on priority.....	29
4.3.4 Created vs resolved	30
4.3.5 Recently created	31
4.4 Retrospectively analyzing the team performance	32
4.5 Implementing to the team processes and user feedback.....	33

4.6 Conclusion	34
5 Use of sources	35
Conclusions	36
Annexes	I
Annex A: Interview results	I
Annex B: User feedback to the productivity dashboard	II

Introduction

Topic definition

In the modern world, agile technologies are becoming more and more popular, as a result, many companies are looking for tools to make their work more efficient with agile sprints. One of the tools is a project called Jira from Atlassian. This project allows us to track work progress in sprints. The variety of instruments could be settled according to the company requirements, everything is accessible from the website, so users just use their credentials and do not have to install anything. The part of Jira is a dashboard creation, this instrument is not fully introduced, so sometimes it is necessary to use external tools to create more transparent and understandable reports and charts. In this work, we are trying to analyse the possibility of creating a useful dashboard using Jira tools.

Aim of the work

This work has a goal to describe different tools of Jira from Atlassian with a concentration on project productivity analysis based on created charts and reports. The main aim is to create a dashboard to analyze different project components' successful work inside a big project, based on the selected metrics. We also review the market of the productivity software to identify potential better fit.

The target audience

This work could be helpful for people using Jira or some similar agile tool to track the development process and to define weak and strong components. People who have just started to work with agile technologies could use the work to get familiar with Jira tools and decide if to use them in their projects. Even though the practical part will be created on data from the large company it would be possible to adjust the process of creation and the dashboard itself to use in a smaller project.

For managers, the result of the work will be suitable to adopt the technology and to have an overview of project success. General workers could use the tool to see their progress and to compare it with other components or departments.

Methods used

To achieve the goal of the research the different sources related to the Jira, agile technology, work productivity metrics, and other topics were studied. Analysis of different tools to

create reports and charts was provided. There are a few manuals from Jira about its tools and functionality, currently the main language of those documents is English.

Work structure

The thesis is split into several parts followed by the conclusion.

The analytical part covers the analysis of the agile, agile teams and their needs, proceeding to the description of the productivity metrics, ways to visualize those, description of the most useful charts for software development teams, and analysis over the competition on the market of Productivity Tracking software; we end up this part with the review of the chosen productivity tracking solution capabilities.

The practical part is dedicated to the process of the selection of the productivity tracking software for the team, designing the productivity dashboard with providing implementation details, retrospective analysis, and user feedback upon using the dashboard in the agile team process.

Work outputs

The analytical part of the work provides the information for the agile teams on the productivity tracking, the value of the productivity dashboard, and the analysis of the current state of the productivity tracking software market.

The practical part provides the guidelines for productivity dashboard design, implementation, and integration into the team scrum events.

1 Analysis of the Productivity Software needs of the Agile Team

This chapter is dedicated to analysing the subject of the research – definitions and methodologies of the agile, agile team, scrum framework, productivity metrics, the various ways to collect and analyse the data to track those metrics, including the visualization with the charts, benefits, and risks of using productivity dashboards and finally review the tasks and main functionalities of the productivity software.

1.1 Agile

First, let's define what **agile** is. Based on the Agile Alliance (What is Agile Software Development?, 2013), one of the most influential organizations working with the agile topic:

Agile is the ability to create and respond to change. It is a way of dealing with, and ultimately succeeding in, an uncertain and turbulent environment.

Accordingly, we can give the definition to the **Agile Team** (Beck, Beedle, van Bennekum and Cockburn, 2001):

An Agile team is a group of employees, contractors, or freelancers responsible for executing an Agile project. Agile teams are typically co-located and often wholly dedicated to the project during its timeline, with no obligations to other projects.

The four **main pillars** of the agile process, which guide us in the productivity assessment and visualization are:

- *Individuals and interactions over processes and tools.* Any strategies or tools should be flexible enough to adapt to the needs of the team, hence the software we select for productivity tracking should be highly adjustable.
- *Working software over comprehensive documentation.* The lack of documentation at the start of the project drives us to the need of finding a solution to quickly generate the productivity information rather than using settled guidelines and instructions.
- *Customer collaboration over contract negotiation.* The incomplete information at the start of the project caused by this principle brings the need to re-estimate the deliverables multiple times based on the performance of the team.
- *Responding to change over following a plan.* This principle dictates the need to constantly review the performance of the team and adjust the plan accordingly and showcase the value of this work.

1.2 Agile frameworks overview

Agile methodology (or even rather a mindset) is purposely defined very broadly to be able to serve the wide spectrum of companies and development approaches, however, when it comes to the practical usage, some of the workflow management frameworks come into play to consult the teams on the approaches and best practices. Some of the most prominent of such frameworks are Scrum and Kanban. In this section, we will provide the definition for those and the overview of the Scrum framework, which we would like to concentrate on in our research.

Scrum is a framework for developing and sustaining complex products (Burrows and Hohmann, n.d.).

Kanban is a workflow management method for defining, managing, and improving services that deliver knowledge work (Schwaber and Sutherland, 2020).

Next, we will look into more details in the **roles and ceremonies** in Scrum, in order to understand at which stages of the process the data about the productivity can be collected, when it can be measured and evaluated, and at what point conclusions can be drawn and actions taken to improve the team performance and who facilitate this process.

Main **scrum team roles** (Schiel, 2010) are:

- Product Owner – the representative of the stakeholders of the project, responsible for the direction of product development and the project progress. Deeply understands the requirements of the project and the perspective of the stakeholder.
- Scrum Master – ensures that the team stays coordinated and supports the progress of communication on the project to make individual team members as efficient as they can be.
- Development team member – engineer, designer, tester or writer, any individual contributor, as teams are usually built cross-functionally. Brings in the skills needed to execute the project.
- Stakeholder – the end user, business executive, or investor – provide direct input to the project and brings in the expectations.

Bigger teams can also include other roles, such as domain experts, independent testers, integrators, or architect owners.

The most used **scrum ceremonies** (McConnell, 2003) (also called scrum events) are:

- The Sprint – event with a fixed length of one month or less to create consistency. A new Sprint starts immediately after the conclusion of the previous Sprint is done. The Sprint employs the following rules:
 - No changes are made that would endanger the Sprint Goal;
 - No decrease in quality;
 - The Product Backlog is refined as needed;
 - The scope may be clarified and renegotiated with the Product Owner as more is learned.
- Sprint Planning – initiates the Sprint by outlining the work to be done during the Sprint and creating a plan based on that. The following topics are raised during the Sprint Planning:
 - What is the value (goal) of this Sprint?
 - What can be executed during the Sprint?
 - How will the chosen work get done?
- Daily Scrum – the meeting to inspect the progress towards the sprint goal and reduce the need for further communications during the day. No changes to the sprint scope should be taken during the daily scrum, so it is not advised to use the productivity dashboard in this meeting.
- Sprint Review – the meeting to inspect the outcomes of the sprint, review deliverables, and determine future adaptation. Usually, the sprint velocity (in story points) is defined in this meeting, followed by the process of identifying the ways to increase the productivity and effectiveness of the team. Sprint Review meeting is the first point at which the team can benefit from the usage of the productivity dashboard, to not only look into the changes on the board but also get the insight into the immediate changes in allocation
- Sprint Retrospective – the meeting to inspect the ways to increase the effectiveness and quality of the software. At this point most of the charts in the productivity dashboard can be used, especially those concentrated on the error rates, feature to bug ratios, amount of time spent on descopeing features, and general burn-down charts.

1.3 Software Team Productivity Metrics

Another important term to review when talking about productivity metrics in agile is **story pointing** the practice of assigning the metric symbolizing the implementation difficulty of a certain user story. Fundamentally, it showcases how challenging a story is for the team based on the efforts, complexity, and risks. Most of the productivity metrics listed below are based on the story points to provide a relative measure of accomplishment within multiple periods (sprints).

“Enterprise-Scale Agile Software Development” (J. Schiel, 2010) deeply researches the topic of the team productivity as the key goal of agile and looks through the various methodologies that allow to enhance every sprint ceremony with the help of the set of metrics created based on the data collected during the development lifecycle and represented in the form of a set of visualizations serving as a material for Product Owners, Scrum Masters, and development teams. Let us review the **key productivity metrics** suggested:

- Achieved DONEness – the percentage of the finished tasks during the sprint that was confirmed as 100% done by the product owner. Lower achieved doneness may signalize the lack of Product Owner involvement in the sprint.
- Velocity – the three-Sprint moving average of the sum of the story points of the user stories completed during one Sprint. It cannot be compared across the teams but can be used to define the progression of one team
- Defects – this metric balances against velocity, to make sure more finished work is not done at cost of introducing more bugs.

Based on the author of “Professional Software Development” (McConnell, 2003) Steve McConnell, we can extend the list with the following:

- Code review turnaround time – quick cycles of the code review mean code not getting unneeded conflicts and being shipped to testing earlier on.
- Pull request size – the smaller pull requests mean better granularity of the task and usually allow to reduce the amount of the defects missed.
- Work-in-progress – too many items being worked on at the same time bring the risk of missing the sprint goal, as the people finishing their tasks will not be able to pick new ones.
- Time-to-open – time to start working on the issue request, the less time, the less damage can be caused by the defect.

Another view on the issue is provided by Aakash Gupta, Head Business Consultant at Saviom, who adds the following to the list of metrics:

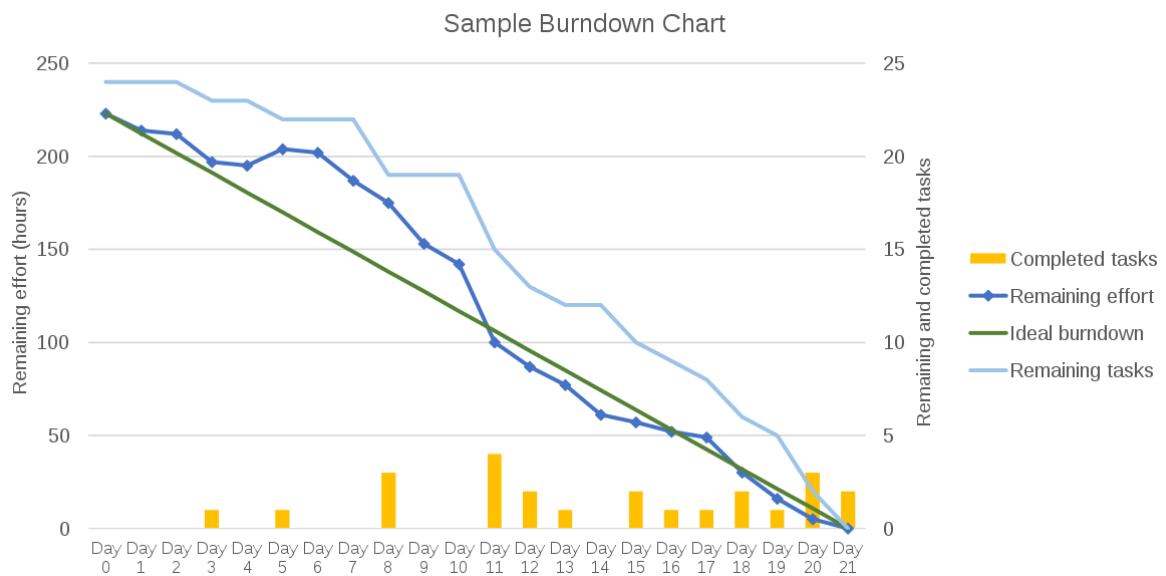
- Cycle time – shorter sprints usually allow to define the scope better and achieve tangible results faster.
- Fully finished stories – the percentage of the committed tasks that were finished during the sprint.

1.4 Productivity Charts

In order to track the productivity metrics, we need to visualize them properly. The data to analyze is collected both before the sprint, with the story points assignment, during the sprint with the story reassessments, and adding in the actual time spent on the issue and after the sprint, during the retrospective by sharing the impressions on the retrospective. A part of this data is rather a qualitative feedback, such as the ideas shared by the team on retro, but another part is a quantitative feedback which can be brought to the form of various charts and provided to the team to review, discuss and act on accordingly. Building the dashboard of such charts is the main point of this work and will be covered in the practical part.

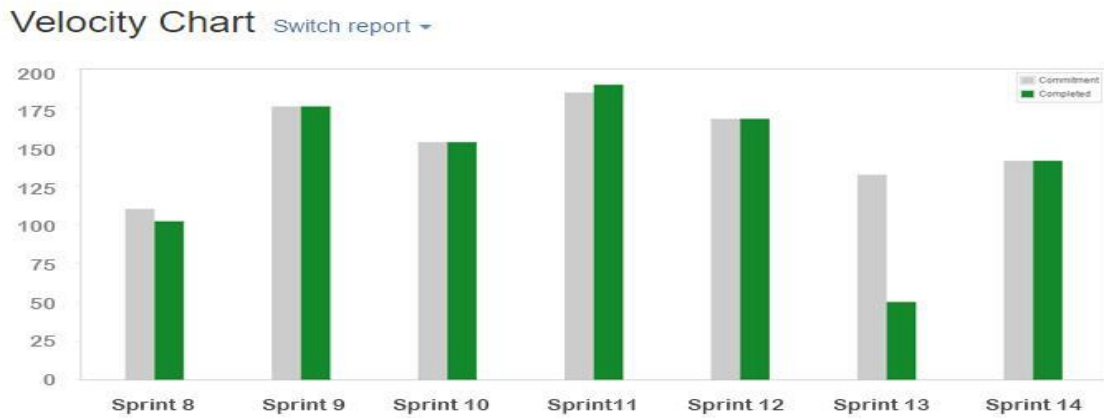
A set of charts that are usually used to display the aforementioned metrics can be split into several groups (Chemuturi, 2013) which are described below and displayed in the Annex 1.

The first group consists of charts displaying doneness and the percentage of the fully finished stories. These charts are best used to identify potential bottlenecks and need to make the adjustment in the team structure or even team participants. The first group consists of Burndown Chart – displays the actual and estimated amount of work done in a sprint, Version Time Tracking Report shows the progress towards completing a given version, based on the logs and time estimates of individual contributors, Version Workload Report displays how much outstanding work is left to finish the given version.

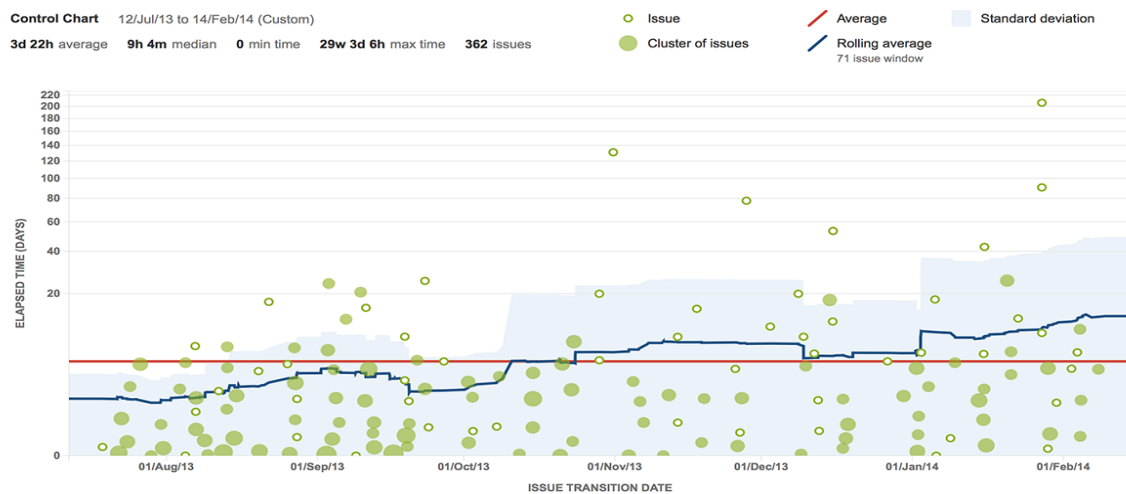


Picture 1 Burndown chart

The second group concentrates on the team velocity and cycle time, allowing to predict the amount of work that can be achieved in the next sprint and the number of sprints to accomplish certain product goals or versions. The reports in this group are: Velocity Chart – shows the amount of value delivered in each sprint, Control Chart, showing the cycle time or lead time for the product (for all the next examples it can be also sprint or version), Cumulative Flow Chart – an area chart displaying work-in-progress, User Workload Report gives the view into how much work a user has been allocated, and how long it should take.

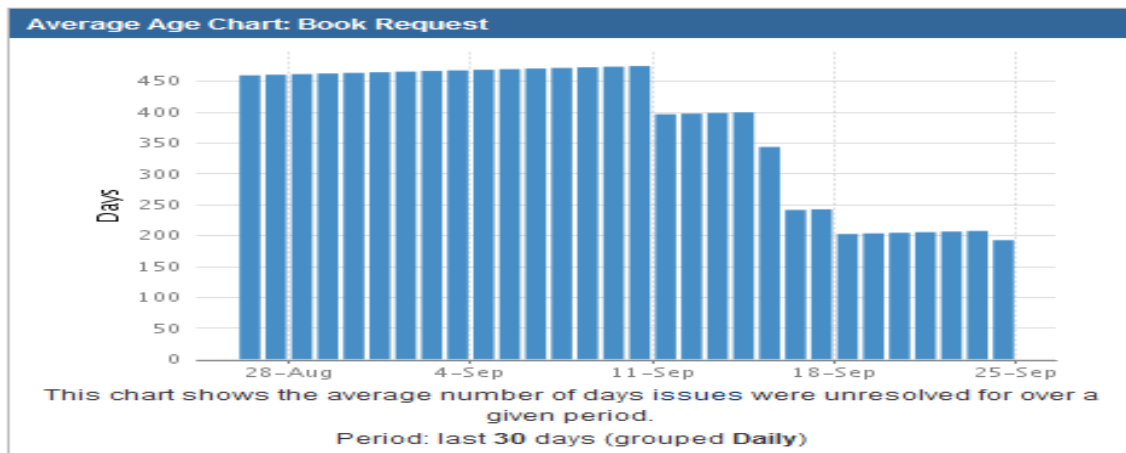


Picture 2 Velocity chart



Picture 3 Control chart

The third group is especially important for observing the metrics related to the defects and backlog cleanness, such as the number of issues and time to resolution, giving insight to the team if they need to slow down or change the processes to improve the quality of the software. Some of the reports in this group are: Average Age Report shows the average age of unresolved issue, Created vs Resolved Issues Report – the number of issues created vs number of issues resolved over a given period of time, Recently Created Issues Report, which displays the rate at which issues are being created, Resolution Time Report shows the average time taken to resolve issues, Time Since Issues Report shows the number of issues, which were started/updated on a given date



Picture 4 Average age chart

Finally, the last group is the charts of general-purpose, which can be used to display any of the metrics above and many of the other custom metrics related to the specifics of the given scrum team. Examples of such charts are Pie Chart, Histogram, Line Chart, Scatter Plot or Box Plot.

1.5 Risks of using the productivity charts

It is also important to outline the risks of using the dashboard representation of the productivity metrics. Based on the “Rethinking Productivity in Software” (T Zimmerman, C Sadowski, 2019) we can outline the following risks.

- Favoring numbers over text – it is often easier to present numbers instead of text, as a result, the dashboard is less likely to give an overview of the features most mentioned in bug reports.
- Losing relevant context – e.g. a dashboard that is comparing the time in the browser versus the time in an IDE might not explain what work has been done there.
- Non-explanatory – the dashboard does not explain the reasons, e.g. why the higher amount of bugs was produced, less time spent coding, etc, the conclusions without the inputs of the team might be flawed.
- Received results depend on what is being measured – as Goodhart’s law says “When a measure becomes a target, it ceases to be a good measure”.
- Low quality of output with low quality of the data – oftentimes the data collected automatically does not reflect development reality and can spoil the outputs.
- Misinterpretation of performance into productivity

1.6 Conclusion

In this chapter we outlined the agile definitions, the methodologies implementing agile and the main processes and events in those, reviewed the scrum team composition and responsibilities of the members of such team. We followed up by reviewing the productivity metrics and selecting the ones most important for agile teams, such as achieved DONEness, cycle time, time-to-open, work-in-progress and fully finished stories. Next, we went through the ways to track the metrics and visualize them, and talked about the pros and cons of using dashboards of charts for that. Finally, we selected the charts that could help the most in visualizing the selected metrics, such as the Cumulative Flow chart, the Control chart, the Average Age chart and the Pie chart for custom visualizations.

2 Analysis of the Productivity tracking software market

This chapter is dedicated to analysing the main competitors on the productivity software market, their offering, typical and exclusive functionalities, methodology to gather the data and build up visualizations, providing insights to the scrum masters and project managers to boost the productivity of the scrum team.

2.1 Productivity Tracking software requirements

With the understanding of the agile processes, productivity metrics, and the ways to visualize them which we gathered before, and consulting with the best practices for productivity tracking (V.A. Johnsson, H. and Kihlstedt, P., 2005) we can now define the requirements to the productivity software we are going to use and it's reporting capabilities.

First and foremost, any productivity dashboard needs a good data source. While it is possible to manage an agile team with the corkboard and a set of papers, writing down the sprint summaries and backlog items on the paper or in an Excel sheet, it becomes unhandy when it comes to creating the data input for the charts. So, the first requirement is the ability to create the tickets, boards, and sprint notes within the software itself and manage daily changes there easily. It is also a big plus if the commenting system for the issues works well and allows to preserve the communication the team had around specific problems.

Secondly, naturally the data we can use to track the productivity and collect automatically is distributed to multiple services: version control system, IDE, communication solution of choice, time tracking service, custom data generated by our own services and 3rd parties – all of this can immensely enhance the reports we build, so an important requirement is either have such capabilities within the software or have a reliable and well-documented process for integration and continuous replication of this data.

The most important part is definitely a tool to create a dashboard with charts itself – a powerful templating system can not only save time building the dashboard, but also give insight into the latest and the most powerful visualizations the team can use. In the previous chapter we discussed the types of charts modern development teams should be most concentrated around when tracking productivity, those should be a bare minimum when choosing a productivity tracking software. It is also implied to have an editor allowing it to create various custom reports.

Finally, while it is not a must, as the productivity software of our choice will not only be used for creating the reports but will also manage the agile development lifecycle, will be integrated with other systems we use and ultimately be a tool the team interacts with daily and on the most of the events, it would be a strong bonus point to be able to handle plannings, scopings and retrospectives within the tool as well as integrate it with our knowledge base to reduce the number of various tools the team needs to learn.

2.2 Productivity Tracking software market competitors

Based on the requirements we outlined in the previous chapter and the research we conducted on the productivity software market, we can outline the main competitors below.

Clickup – all in one generic tool for project management, meant to cover not only productivity tracking but also various aspects of product development and team communication:

- Rich tools for organized collaboration and reporting.
- Features for organizing calendars, managing team capacity, scheduling time, and milestone tracking. All synced with Google Calendar.
- Smooth integration with other productivity tools and time-tracking software.
- High customizability of spaces, workflows, tags, features, and branding.
- On the downside, a rather overloaded interface, which might be more difficult to pick for beginners.

Monday – best suits small and medium companies. Binds employees to work processes, and personalizes deadlines, timelines, and progress tracking for individual contributors to increase their involvement. The main features Monday promotes are:

- Ease of usage and starting up, by choosing a template (exist for many types of projects) and customizing it – configuring columns to track time and display responsible parties).
- All important sheets, checklists, files, briefs, and conversations are collocated, reducing the need in other communication ways.
- Allows to store all the project-related assets, like files, timelines, calendars, Kanban / Scrum boards.
- Elaborate mobile application, winning over competition and allowing the stakeholder to take part in the process more actively.
- Wide variety of services to integrate with – Dropbox, Excel, Google Calendar, Google Drive, Integromat, Slack, Trello, and Zapier.
- On the downside – lower capabilities in the reporting section, part of the metrics are meant to be reviewed in a way other than classical charts.

Jira – suits well to larger companies, allows the most elaborate reporting and customization not only of the workflows but also of the data collection. The list of Jira's main features includes:

- Providing both a customizable Scrum board and flexible Kanban board to help team members stay concentrated on delivering without delays and allow different modes of work based on what works better for the concrete team, ability to switch between those rather easily. Ability to create customizable workflows matching the work of your company.
- Road-mapping capability to be able to outline the long-term vision and keep stakeholders informed about the plans and link it to the current work the team is executing upon.
- The most elaborate system of reports and templates among the competitors, with the ability to create fully custom reports and provide different permissions to the contributors to view their performance in every period. The support for the real-time reports together with the insightful tips.
- Working with custom filters using Jira Query Language (JQL) to be able to drill down in both the board and reports and get to the root of any productivity problem.
- Well integrated with developer tools and version control for tracking individual contributors. Issues can be connected to code, feature flagging, and a full overview of the development process.
- Around 3,000 of other apps which can extend and optimize the workflows.

2.3 Conclusion

There are also a number of tools of the second echelon, which feature similar capabilities but have a lower presence on the market or a slightly different purpose. Some of those that we met the most when doing the research are Nutcache, Trello and Asana - those tools were added to the market share comparison table, which can be found below, but were left out of the detailed review in this chapter.

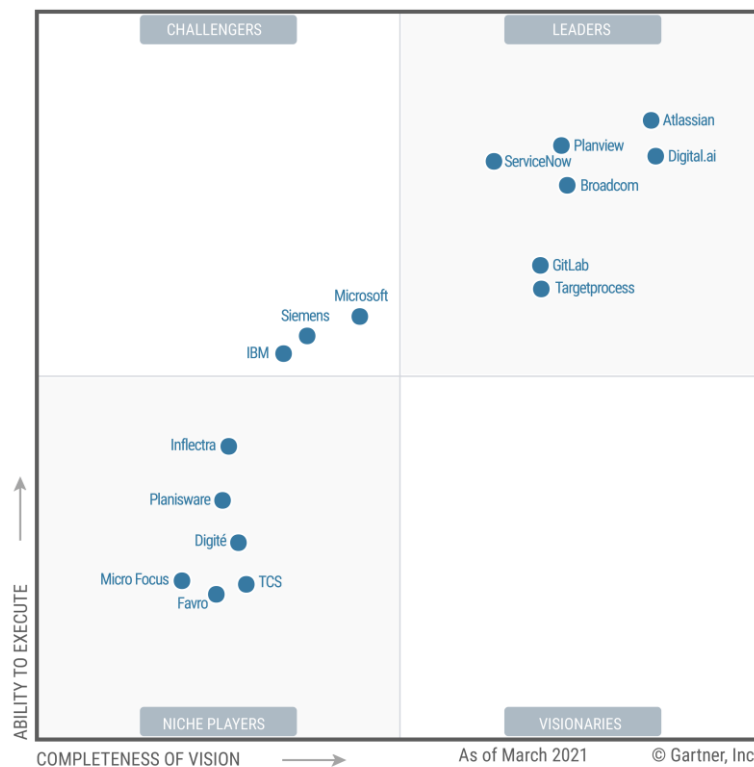
Table 1 Analysis of the productivity software competitors by market share

Competitor	Number of projects	Market share
JIRA	51,570	44 %
MS Project	20,963	21 %
Monday	4,321	5 %

Asana	3,258	4 %
Trello	3,135	4 %
Clickup	680	1 %
Nutcache	320	0.5 %

We also reviewed the Gartner magical quadrant visualization, which helps to identify how the product manages balancing of the forward-thinking with serving the needs of the current market and identified Atlassian (JIRA's creator) among the Leaders group, scoring higher than the competition.

Figure 1: Magic Quadrant for Enterprise Agile Planning Tools



Picture 5 Gartner magical quadrant for enterprise agile planning tools

Based on the assessment we executed and in coordination with the team survey of the most needed dashboard capabilities (you can find the description of the interview process in the practical part and see the outputs and conclusions in Annex A), we decided to stick to using JIRA as a productivity tracking software. The main reasons for this choice were the fact that we have a large company with more than 10 0000 workers and the whole company is using Jira. In our team we have to align with company requirements and also even for us Jira is

suitable, because of its flexibility and a large number of report templates, good integration with the applications we use in our team, and coverage of all the functionalities we scoped out as the requirements in the previous sections.

3 JIRA: tool overview and capabilities for agile teams

This chapter is dedicated to the detailed review of the main functionalities of JIRA – one of the leading competitors in the productivity software market. We will look into the implementation of the ticket management, project management, road mapping and most importantly reporting part.

3.1 JIRA functionality and intended usage

JIRA was developed by Australian Company Atlassian and its path started back in 2002. The main functionalities of the software include the project management, issue tracking, bug tracking, and reporting.

JIRA's main competitive advantage is configurability – with changing the JIRA scheme, it is possible to fully adjust permissions, notification settings, issue fields, and types, but also screens, boards, and workflows on those.

Based on the documentation provided by the vendor (Confluence.atlassian.com. 2017), the process of the team working with JIRA starts with creating a project, a board based on the team development methodology – usually Scrum or Kanban – a list of issues to implement, defined by the Product Owner and added to the team backlog. Based on the board type, issues appear on the board within certain intervals and based on the team's agile processes. The tickets go through the set of statuses (workflow) defined by the board owner, typically through TODO => IN PROGRESS => DONE line, usually with some intermediate statuses for code reviews, testing, etc.

An important part to review for the purposes of our research is a set of ticket types in JIRA, the following default types are exposed:

- **Epic** – usually a bigger initiative taking more than a sprint, the time-to-finish of an epic and the number of issues related to the epic are the main characteristics used in the reporting.
- **Story** – the unit of work for the development team, usually stories describe the whole functionality which might need the involvement on both backend and frontend sides and velocity reporting is predominantly based on the number and size of finished stories during one sprint
- **Task, Subtask** – single pieces of work to prepare during the sprint, typically created during the refinement session. It is considered a good practice to not estimate tickets of this granularity level.

- **Bug** – the issue discovered during testing or in production. The bugs can have different severity and possess their own story point estimates and factual post-execution values. Most of the reports from the Defects category are based on this type of issue.

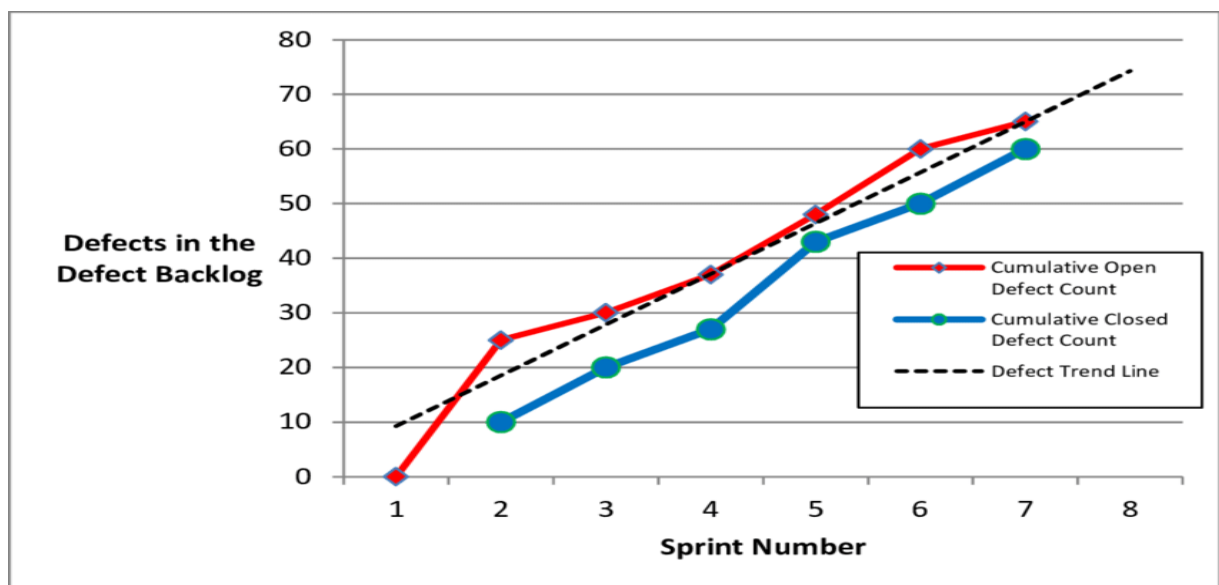
JIRA allows integrations with multiple services, so correctly configured board will have automated update of the tickets within the workflow (getting to in progress, code review, testing statuses), integration with the knowledge base (Confluence from Atlassian is usually the tool of choice), time tracking tool and more. With a set of plugins, it is also possible to organize sprint planning, retrospective, and daily scrum meetings right inside the application, using features such as planning poker, quick filtering and search of the meeting, and sprint notes taking.

3.2 Productivity Reporting in JIRA

To track the progress in Agile, various reports might be used and the effectiveness of reports varies significantly based on the methodology selected by the team. Most of the charts described in the previous section are supported by JIRA and support creation from the template.

Some of the most useful reports for the Scrum team include:

- Sprint Report – provides the view of the sprint progress and priorities.
- Velocity Chart – visualizes “velocity” for the Scrum team.
- Version Chart – burnup chart, showing the project scope changes along with team progress.



Picture 6 Burnup chart

On the other hand, Kanban-driven teams will mostly benefit from using:

- Cumulative Flow Diagram – helps to set expectations on when work will be delivered.
- Average resolution Report – helps to identify “velocity” for the Kanban team.
- Created vs Resolved Report – helps the team to identify if it is keeping up with the incoming requests and extend the team or reduce the scope.

It is also possible to configure a scrum board to be displayed along with charts on the dashboard, changing the regularly displayed columns, swimlanes, quick filters, card colors, and more. Besides, the user can set filters on the issues, such as date, component, priority, and resolution.

Several limitations building a dashboard in JIRA brings. There is no simple way to combine columns from two reports to build a single Custom Report. At the moment the only possible way is to export these Jira Custom Reports to Microsoft Excel and edit there. Besides, only basic integration capabilities with popular reporting applications (for instance Power BI) are available, thus syncing and combining the data with other data sources, for example, operations details, and visualizing everything in a single report can be really problematic.

3.3 Conclusion

To conclude, we reviewed the main tooling JIRA offers to the agile teams, the main workflow of the data collection and the options for data visualization in JIRA, explored the charts suggested for the teams working in scrum and kanban methodologies, additional capabilities and limitations of the reporting.

4 Practical Part: building productivity charts

4.1 Describing the use-case

In this chapter, we are going to describe the work process of the team and the different Jira functionality we use in the day-to-day work. The team is a part of the rather big company with a lot of workers all over the world, with a lot of different departments. While the company uses the agile development process with different methodologies, the QA department is using Kanban technology in Jira. The team is running on sprints, but also continuously tracks planned, in progress, and done issues. We are working in the functional integration team. Our main purpose is to integrate different components together and test the functionality use cases across all of them. In our team, we do not use any agile techniques, but we are using Jira as it is the main tool used by our department to track work processes.

The most common tools we are using in Jira are Epics, user stories, tickets, containers, DCCRs, sprints, labels, and filters. Dashboards were not applicable for our team, which was one of the reasons why we decided to add this tool in our daily routine after the short research we made inside the team and between a few developers' teams (see results in Annex A).

Epics – are the main documentation form; they can cover several components and are delivered during the period of time more than one sprint. Epics are distributed between all stakeholders and require approval from all of them. Once an epic is approved, the user stories could be created.

User story – is a description of the functionality delivered by the precise development team. A user story should be delivered inside one sprint but could stay in progress for a long time. Every new functionality should be described in the epic and then delivered with user stories. The user story could be assigned to only one epic, and an epic could contain more user stories and tickets. Epics could be linked one to another if the functionality they describe is related. Sprints usually contain only user stories and tickets.

The ticket – is the description of a reported issue. The ticket could be created for a newfound issue, entered as a follow-up for the user story or any other issue. Tickets could be linked one to another and to user stories or epic as well. Tickets could be created with different priority and after the analysis, the development team decides in which sprint it will be fixed.

One sprint is a two-week period of time. Each sprint is delivered with a container – there are all finished user stories and fixed tickets inside. When some ticket requires a hotfix, a separate hotfix container is created. Every component delivers its hotfixes separately, contrary to the main sprint container where they are all delivered together.

Let us describe the testing process inside the department:

- The first level of testers is developers. Every component tests their part separately right after it is ready. They have both automation and manual tests. Usually, developer's team has one tester. Sometimes every developer is responsible for its commit and should test it by himself, which could cause a test quality issues.
- We are on the second level and it is a functional integration test team. All components send their parts of the sprint in one container and we install them all together to test their compatibility and an ability to work simultaneously. The team includes two parts: technical and functional. Technical testers are testing integration from a technical perspective. They provide mostly automation tests e.g., live performance, stress tests, and others. Functional testers are responsible for a functionality check. They are going through the documentation and comparing specifications and actual implemented functionality. There are a lot of details and changes with every release, so automated tests are hard to be implemented from a functional view, functional testers are mostly working manually, but the automation is in progress.
- The next level of testers is acceptance. They receive all components together as functional testers, but also have connection with external components, which are developed separately, but the communication between them and our environments need to be tested as well. Acceptance testers should check again how the actual implementation is in alignment with the specification, and users' expectations. They are allowed to approve features for production.
- The last level is a simulation test, where the environment is the most alike with real production and the performance of the product could be tested on the real production data.

Only after that if all tests are passed the feature can go to production.

4.2 Selecting the visualizations

In this chapter, various types of chart which were selected to be used in the final dashboard will be described, as well the reasons for every chart selection will be specified. As it was mentioned before, the short survey between the team members and a few developers was done. The survey was productive, and a few very useful advises were gotten. After the survey was finished the draft dashboard was created.

Annex A references the results of the interview conducted among the teams. In the interview, we were trying to identify the most important metrics and visualizations to build from those we researched previously and get insights into other charts team members would benefit from. In order to properly rank the options by the importance we assigned multiple votes to each respondent so that they can identify the importance of each metric for them - this way one could give one vote to every option to show they are equally important or give all the votes to the single option if it is very important to him or distribute the votes in the other manner.

From the interview, we made a conclusion, that for our colleagues the most important metric is the ability to track ticket lifecycles, check how many bugs were found inside one delivery, and also see personal performance for each team member. Less popular were severity metrics, possibilities to have a view of next-level testing and how many bugs could be caught after our team, period of time from the idea to implementation, and changing of tickets' statuses inside one sprint. Those last metrics are more important for developers and managers, but not for the majority of our team members. During the research, we found out that our team is not using estimated time. In the company the estimated time is applied only for the developers' teams, as a result, we were not able to create a few types of charts, e.g. velocity charts. On the other hand, we could create an Average Age Chart and Control Chart, which are suitable for the Kanban methodology used in our team.

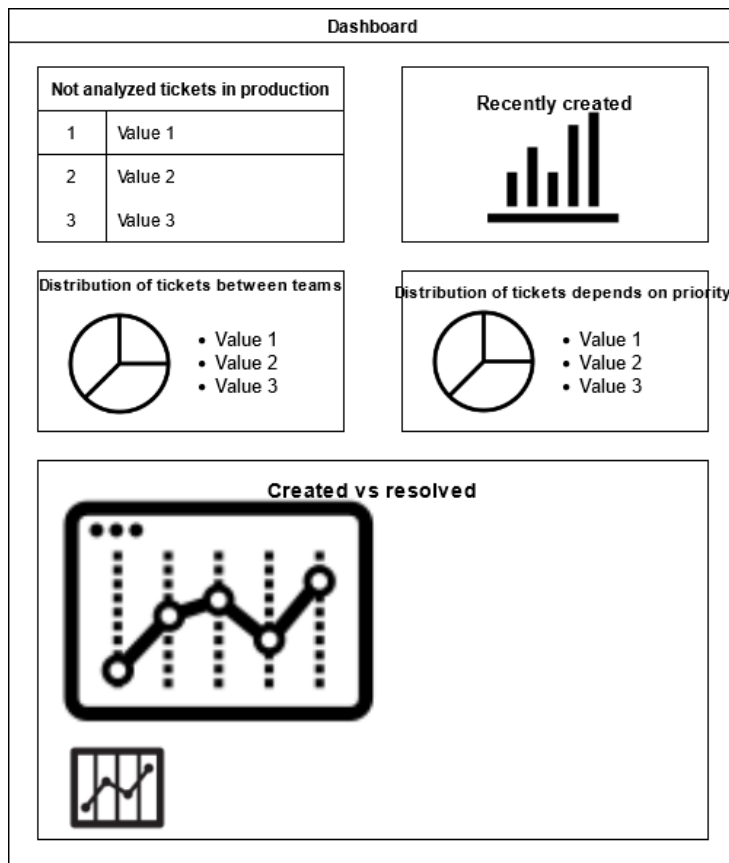
Another limitation which was detected during the practical preparation of charts, was Jira user roles and access rights. We did not have access to rich filters creation functionality, so some complicated charts also were excluded from our options. There was a possibility to use already existing filters but did not find any suitable for our purpose.

In this work, we have a goal to create a Jira dashboard, which could be helpful for our team productivity. We decided to select one precise date when a sprint goes to production and take all not-closed tickets for this date. From this point in time we wanted to follow the relation between components and their productivity, which is shown in a number of fixed tickets, to check on which testing level most of the tickets were created and to check tickets prioritization. According to the short survey we made between the team and colleagues, those graphs could be helpful to determine which component or team needs some support in their work. As a bonus for our team, we made a chart to see how many and which tickets were created for the current sprint by our team.

Based on the needs of the team, we selected to create 5 charts to view:

- Tickets that are not analyzed and the sprint they have been occurred in is already in production;
- The correlation between teams created tickets in the observed sprint;
- The correlation between prioritization of tickets and their amount in the observed sprint;
- The correlation between created and resolved tickets in the selected project
- The list of tickets created by our team members in the observed sprint.

The draft dashboard was drawn to imagine how it should look and to present it to the team. The draft was approved and then we stepped to the next stage.



Picture 7 Draft dashboard

4.3 Implementation description

In this part, we will describe the whole dashboard creation process, including possible problems, data issues, and the process of preparing filters to create suitable charts. As it was mentioned in the previous part we decided to create 5 charts, let us describe them one by one. The first chart creation process will be described in detail step by step and for the others, we will show a filter and a result chart.

4.3.1 Not analyzed tickets in production

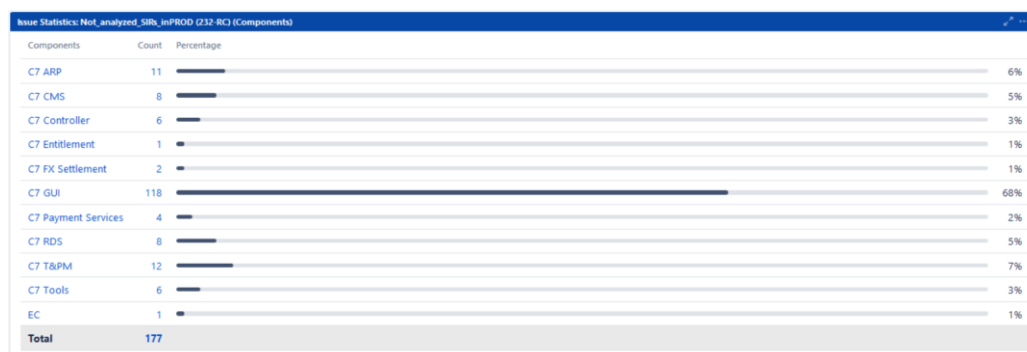
The first chart has a goal to display tickets that are not analyzed and the sprint they have been occurred in is already in production. As started parameters, we have that currently in the production there is a 232 delivery, a project in which all tickets are created is "Ticket", statuses which we are interested in are "Submitted" and "Analysis started". We are interested only in tickets that are created before the production's delivery, so we need to exclude all tickets with occurred configuration more than 232 from the filtered results, in case if the configuration is not settled in the ticket we are taking as the parameter a day at which the ticket was created. The created date could be before 29-03-2020 for issues that occurred in development and before 12-04-2020 for all other environments as we have two weeks sprints and developers could enter tickets earlier than any other level. To have the

filter results more readable we will order them by the creation date in ascending order. In Jira, there is a possibility to display precise columns in the filter results, which remains the same for all inquiries for the current user. All this we need to set in the inquiry line, there are two ways how to set it by selecting manually from parameters or by writing filter in Jira's inquiry language. Manual selection could be suitable for users who are not familiar with inquiry rules and want to have it more readable. We wrote our filter with an advanced filter option to have everything under control and have a possibility to set parameters as we want.

A possible filter could look like this:

project = "Ticket" AND issuetype = SIR AND status in (Submitted, "Analysis started") AND "Occurred in configuration" is EMPTY AND createdAt < "2020/03/29" AND "Occurred in environment" = Development OR project = "C7 Ticket" AND issuetype = SIR AND status in (Submitted, "Analysis started") AND "Occurred in configuration" is EMPTY AND createdAt < "2020/04/13" AND "Occurred in environment" != Development OR project = "C7 Ticket" AND issuetype = SIR AND status in (Submitted, "Analysis started") AND "Occurred in configuration" != 233-RC AND "Occurred in configuration" != 234-RC AND "Occurred in configuration" != 235-RC AND "Occurred in configuration" != 236-RC AND "Occurred in configuration" != 237-RC ORDER BY created ASC

After the filter is successfully set and results are corresponding with our purpose we need to save it with the “save as” button and also make it visible for others, so anyone will be able to see the chart's result. In case we keep the filter private, it will not be possible to display a chart properly to anyone except us. Once the filter is saved we can start a dashboard creation process from the dashboard menu. In this case, we selected an issue statistics chart, which can display all our components and the number of issues that are not analyzed as a result of our filter.

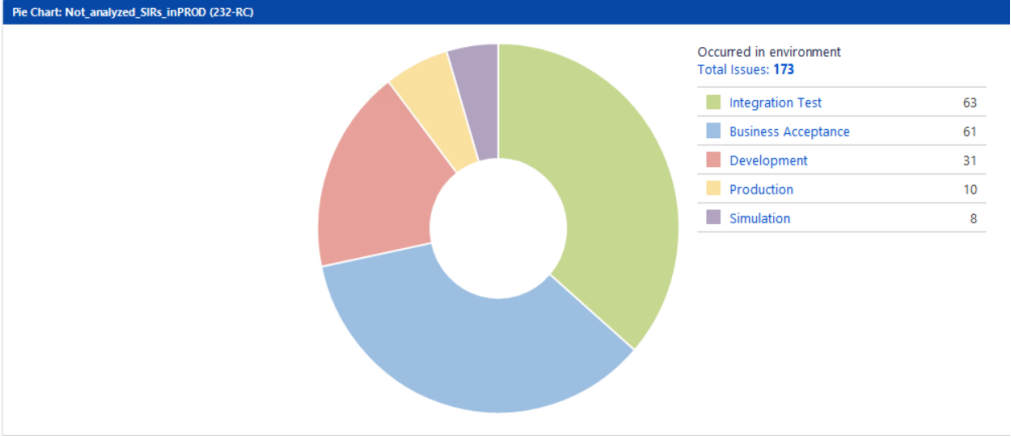


Picture 8 Not analyzed tickets in production

4.3.2 Distribution of tickets between teams

The second chart has the purpose to display the correlation between the initiating team and amount of created tickets in the selected sprint. In this case, we required again to select all tickets created to the concrete date or sprint and also then divide them by the environment they were occurred in. The created filter was not much different from the filter we use for

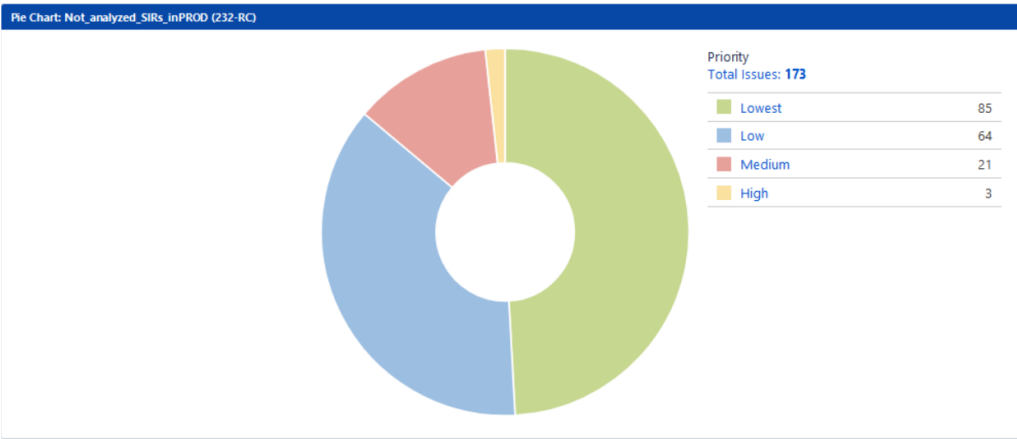
the first chart. Here is an example, how Jira allows using already saved filters to create more charts. The same is for the rich filter, which we were not able to create or modify, but could use already created filters. The disadvantage was that we did not find any rich filter, which could have been suitable for our purpose. For this chart, we decided to make a pie chart, where the difference between initiating teams will be more visible.



Picture 9 Distribution of tickets between teams

4.3.3 Distribution of tickets depends on priority

To have more information about created tickets for the third chart we decided to display the correlation between the ticket prioritization and the number of the tickets created for a certain sprint. We use the same filter as for the previous chart, and pie chart, but now we were interested in the issue priority parameter. With rich filter functionality, it could be possible to join two charts together and to track the real correlation between initiating team, ticket priority, and time which was required to solve the ticket. Unfortunately, with our privileges we were not allowed to make such a chart.



Picture 10 Distribution of tickets depends on priority

4.3.4 Created vs resolved

The forth chart shows the correlation between tickets created and resolved on the same day during the selected period of time. For this chart we use a classic filter through the project, so basically in the chart maintenance window, we just selected a project name and set other settings. This chart is one of the simplest to create, but informative. In the Annex 3, it is displayed the edit chart window. There is possible to select a filter or project to display concrete issues. The next parameter is a period of how often we want to get filter results, we selected weekly to have a view per week, as we have two weeks sprint we can track at which week issues are created or resolved. Then the number of days goes, the maximum there is just 300, so it will not be possible to display this chart for a whole year, but it is usually used for a short perspective view. We selected 180 as half a year, here was possible to display only data till the current date, because the filter with a separate time period limit did not display anything in the chart. We made a conclusion that this chart template has a goal to display current, but not historical data.

Project or Saved* **C7 Ticket**

Filter:

Project or saved filter to use as the basis for the graph.
[Advanced Search](#)

Period* **Weekly** ▼

The length of periods represented on the graph.

Days Previously* **180**

Days in the past to collect data for the selected period.

Collection Operation* **Count** ▼

Progressively add totals (1.. 2.. 3), or show individual values (1.. 1.. 1).

Display the Trend of Unresolved **Yes** ▼

Show the number of unresolved issues over time in a subplot.

Display Versions* **All versions** ▼

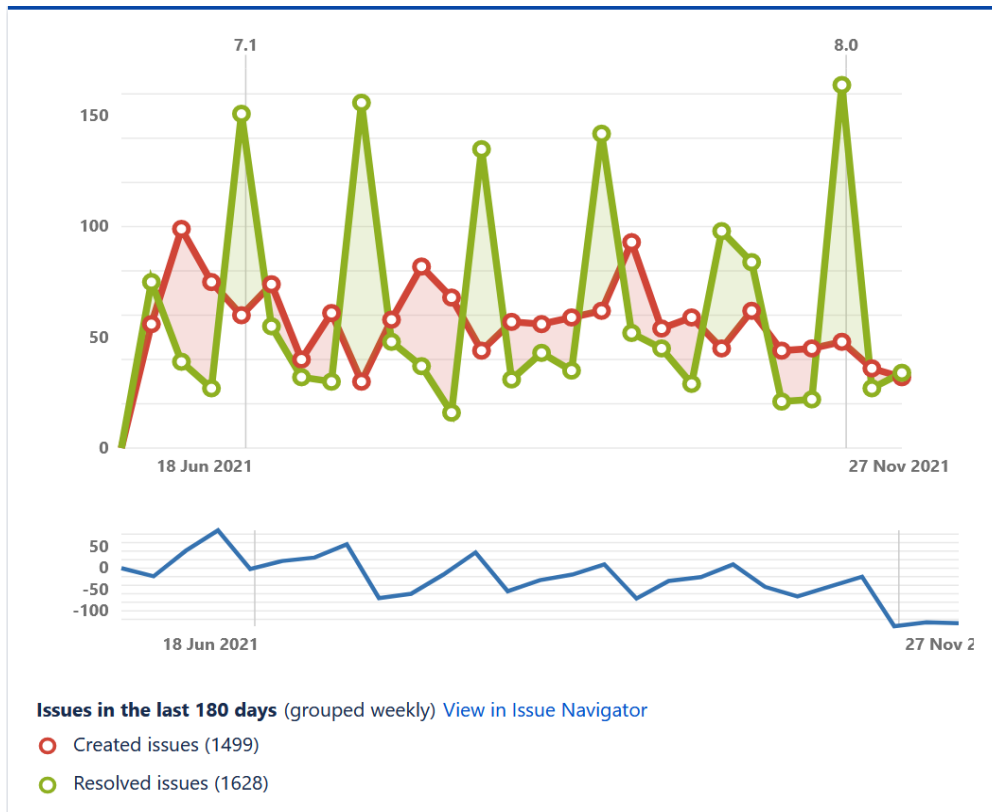
Show when versions were released on the chart.

Auto refresh ☐ Update every 15 minutes

Save **Cancel**

Picture 11 Created vs resolved - edit

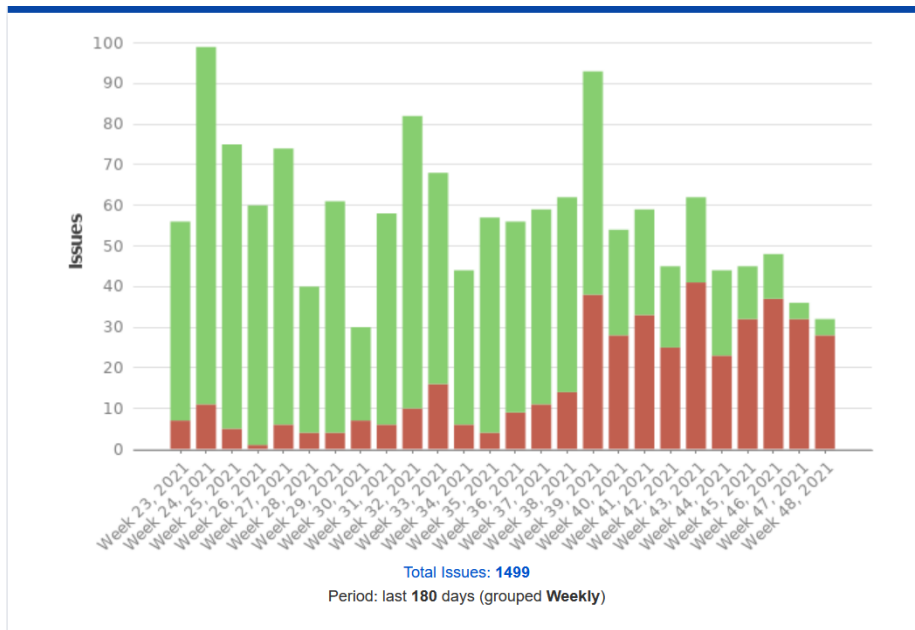
This chart also allows to include a trend graph, which displays a difference between created and resolved tickets in observed date or period of time. In this graph, we can observe that during the observed time number of resolved tickets constantly became more and more than created, which could be taken as a good sign. From the main graph, it is also visible how many tickets are resolved right before the new great version release, e.g. 7.1 or 8.0.



Picture 12 Created vs resolved

4.3.5 Recently created

Recently created issues chart was selected as the last one we describe in this work. The graph displays a total number of recently created tickets during the selected period of time, e.g. weekly. In this visualization we can track how many tickets were created during the period and how many of them we closed. At this point, it is important to mention that closed tickets mean tickets in a final state, which could be whether installed or rejected. This chart does not have a proper legend, so we need to clarify that each column displays the total amount of created tickets, and red means the number of tickets that are still not closed. The creation process for this chart is also quite simple, we just select project or filter name, period of time we want to group data by, and period by which we want to display data.



Picture 13 Recently created

4.4 Retrospectively analyzing the team performance

In this part, we will describe what we saw on the created charts and in which situations it could be useful. We will go through all charts described in the previous part which comment on what we can take from the displayed results.

From the first chart, we can make a conclusion, that the component, which needs the most support or help is a GUI. In comparison with other components, it has almost ten times more open issues. On the other hand, GUI bugs are usually the smaller ones, which could be ignored by users for a very long time. This could be taken to the next management level, to initiate an improvement of the ticket-fixing process and also is helpful for our team to understand what to expect when we create a ticket for the precise component.

From the second chart, we can see that the majority of the tickets are created by Integration and Acceptance testers, which could be a good sign as those are main test levels and they are specialized in bug-finding. Still, we have a few tickets from production and simulations, which is a classic situation, because still not all issues could be caught before the delivery goes on the higher levels. To decrease the number of bugs the various measures could be applied, such as improving the quality of the output of the development team by introducing more elaborate processes or increasing the number of testers.

From the third chart, we can conclude that the majority of tickets are created with the low or lowest priority, there are only 21 medium issues and 3 high. In our organization, we use priority to detect how critical the issue could be for a customer. The lowest priority is for small issues, usually on the GUI side, which could be there for a long time and not bother users much. We have the lowest priority set by default, so sometimes when the creator does not change it even critical issues could be marked as lowest, but they are usually fixed faster

because of background communication between developers and ticket initiator. The low priority is the most popular for non-critical issues when functionality does not affect customers much or it could be some enhancement which will make some activities more effective. Medium priority is for issues that could be a potential blocker, but there is some bypass to make the functionality work without fixing the issue. As bypass could be a DCCR entered for higher environments, some requests which could be sent from a different window or from a backend instead of GUI. Those issues are usually fixed during two sprints and should not happen for a customer. High priority issue is a potential hotfix, which could block a functionality completely without any bypass. Those issues should be fixed immediately to avoid problems on the next levels.

Forth chart is one of the most informative as it was mentioned before. From the chart we can conclude that developers strive to resolve a great number of tickets till the end of every month, this is visible on the peaks of resolved tickets which repeat every four weeks. At the same time amount of newly created tickets looks completely independent from any releases or sprints, as the red line separately does not have any observed trends. On the other hand, this line does not have great fluctuation and the maximum of created tickets does not go higher than 100 tickets.

From the fifth chart we that something critical happened on the week thirty-eight of this year, as the number of the created tickets grew really fast and the difference between wicks is about 30 tickets per week. On the other hand, the number of unresolved tickets also grew fast for about 25 tickets. We can conclude that developers continue to resolve approximately the same number of tickets, but they were created in much greater amounts. There was a beginning of the fall, so one possible reason could be that more people returned to work from vacations. Another possible reason could be a new great functionality which was delivered and required more testing and as a result, more tickets were entered.

4.5 Implementing to the team processes and user feedback

Once the dashboard was ready, it was time to provide it to the team and the main stakeholders to gain insights from it and identify the place for the potential improvements. In order to be able to use the dashboard in the team agile ceremonies, it is important to present it to the scrum-master or agile coach - in our case the team lead is playing this role. We demonstrated the dashboard to them along with the results of the retrospective analysis of the team performance over the last 6 months and got the buy-in to use it on the ceremonies during the next sprint.

We used the visualizations prepared on the sprint review, sprint retrospective, and sprint planning meetings and decided against using it on the daily scrum meetings, to keep those as short as the team is used to.

During the sprint review meeting we demonstrated the outputs of the team and, along with reviewing the done column in JIRA and discussing which of the most important issues we were able to identify before they got to the production, we reviewed the charts displaying

the amount and severity of the issues we tested overall, comparing it to the number of the bugs found and numbers of issues and bugs processed on the other testing levels.

During the retrospective that was held right after the sprint review meeting and was mainly based off the results of the previous sprint, we discussed the problems the team is facing recently because of the large number of issues coming into testing without finalized documentation, hence the scope is not clear, and we spend a lot of time figuring out the correct behavior of the system. This observation was supported by the Issue Statistics chart and after the discussion, we created a follow-up item to resolve the situation with the product team.

Finally, during the planning for the next sprint, we used the outputs of the Average Age chart to identify the most important issues to concentrate on at the start of the sprint and updated their priority in JIRA as needed. We also provided the stakeholder with a generic estimate of what can be accomplished during the sprint based on the Control chart.

After using the dashboard for the whole sprint and receiving positive feedback from the team and the tech lead, we conducted the survey to assess the attitude of the whole team to the new tool we use in our agile processes and identify what parts do they find the most important. The results of the survey can be found in Annex B.

4.6 Conclusion

In this chapter the process of productivity dashboard creation was described. We made an analysis of teams' needs, then the draft dashboard was created and discussed inside the team. After the draft was approved, we created the dashboard and described the process of separate chart creation step by step. We included pictures of created charts in the according chapters, but we did not include the final dashboard, because the picture could be too large and all details will not be visible.

5 Use of sources

Agile Alliance. 2013. *What is Agile Software Development?*. [online] Available at: <<https://www.agilealliance.org/agile101/>> [Accessed 5 December 2021].

Beck, K., Beedle, M., van Bennekum, A. and Cockburn, A., 2001. *Manifesto for Agile Software Development*. [online] [Agilemanifesto.org](https://agilemanifesto.org/). Available at: <<https://agilemanifesto.org/>> [Accessed 5 December 2021]. Beedle, Mike. "Enterprise Scrum". Retrieved 25 September 2015.

Burrows, M. and Hohmann, L., n.d. *Kanban from the inside*. 1st ed. Seattle: Blue Hole Press. Seattle, ISBN 978-0-9853051-9-2.

Adkins, L. 2010. *Coaching agile teams: A companion for ScrumMasters, agile coaches, and project managers in transition*. Addison-Wesley Educational.

Schwaber, K. and Sutherland, J., 2020. *the official Scrum Guide*. [online] [Scrumguides.org](https://scrumguides.org/). Available at: <<https://scrumguides.org/download.html>> [Accessed 5 December 2021].

Schiell, J., 2010. *Enterprise-scale agile software development*. 2nd ed. Boca Raton, FL: CRC Press. ISBN 978-1-4398-0321-9.

McConnell, S., 2003. *Professional Software Development: Shorter Schedules, Higher Quality Products, More Successful Projects, Enhanced Careers*. 1st ed. Boston, Mass: Addison Wesley. ISBN: 0-321-19367-9.

V.A. Johnsson, H. and Kihlstedt, P., 2005. *Performance-Based Reporting: New Management Tools for Unpredictable Times*. 1st ed. Hoboken, N.J.: J. Wiley. ISBN: 978-0-471-75194-6

Storey MA., Treude C., 2019 *Software Engineering Dashboards: Types, Risks, and Future*. In: Sadowski C., Zimmermann T. (eds) *Rethinking Productivity in Software Engineering*. Apress, Berkeley, CA. https://doi.org/10.1007/978-1-4842-4221-6_16 1007/978-1-4842-4221-6

Chemuturi, M., 2013. *Mastering IT project management*. J. Ross Publishing. 978-1604270785.

Confluence.atlassian.com. 2017. *Getting started with JIRA Core | Jira Core Data Center and Server 7.3 | Atlassian Documentation*. [online] Available at: <<https://confluence.atlassian.com/jiracoreserver073/getting-started-with-jira-core-861255635.html>> [Accessed 5 December 2021].

Zimmermann, T. and Sadowski, C., 2019. *Rethinking Productivity in Software Engineering*. 1st ed. Mountain View CA: Apress open. ISBN: 978-1-4842-4220-9

Conclusions

In the modern world, one of the top priorities of every company striving for success is increasing the productivity of the team, which can bring better outputs than extending the staff numbers. The agile approach productivity gains are achieved through the effective communication and high responsiveness to customer demands. Productivity tools make it easier to set goals, track progress, assign tasks, create reports, and communicate with the employees in the real-time. Communication tools, reporting, and scheduling ensure the effective development of the team as well as productive cooperation with the stakeholders.

In our work we have goals to describe Jira functionality and to make a review of a productivity software market. We reached those goals in the theoretical part. As a main body of this thesis, the agile, agile teams, different agile methodologies, productivity and its metrics and various visualization options, as well as the risks connected with using them were analyzed in the first part of the thesis and followed up by the analysis of the market of productivity software. We compared the options and came up with the conclusion on the leading competitors on the market and the best option for our team.

The main goal of our work was to create a dashboard in Jira and analyze work productivity of different components in a large company. In the practical part we described the process of selecting correct visualizations for the team, implementation of the dashboard in JIRA, retrospectively analyzed the performance of the team, implemented the dashboard to usage in the agile processes of the team for one sprint and finally collected the feedback to the dashboard from the perspective of different scrum roles. As a result our goal was reached and now we have a more clear retrospective of our department productivity.

In the future the thesis could be extended with implementation of different dashboards with more charts described in the analysis part, researching the long-term impact of the productivity dashboard on the scrum team, creating the productivity dashboard in another software, like Monday or Clickup and comparing the traits and the output of the teams.

Annexes

Annex A: Interview results

Question 1: “We are building the productivity dashboard to track the team performance. What capabilities should it have? Distribute 7 votes between the most important options, based on the priority”.

Question 2 (optional): “What other capabilities should the dashboard have?”

Total number of respondents: 22 (12 female, 10 male).

Age of respondents: 21-51.

Table 1 Question 1 answers distribution

Answer	Votes	Rank
Display average age of the opened issues	44	1
Display the number of bugs caught on each stage	31	2
Display personal performance of the team members	23	3
Display the number of issues processed by severity	16	4
Identify the number of bugs not caught after our testing stage	14	5
Display overall lead time - from the idea to production	10	6
Display the ticket statuses of the tickets worked on during this sprint	6	7

Question 2 answers distribution:

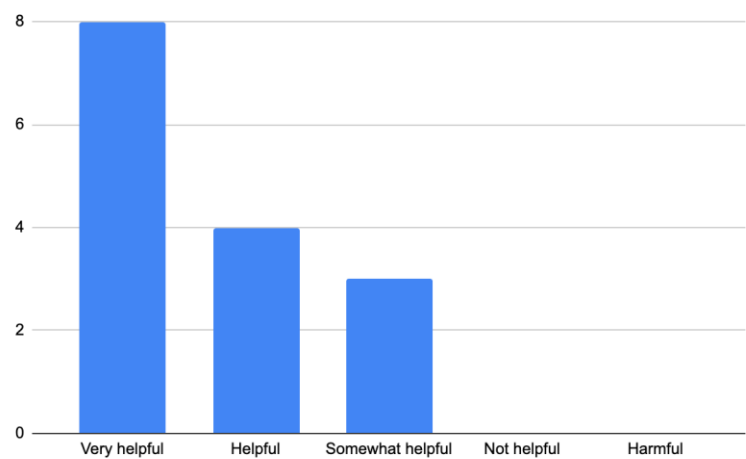
- “Would be good to show the amount of bugs not fixed by the developers”
- “I want to see how many my tickets are hotfixed or rejected”
- “You can add the report with amount of time off and educational activities taken by the team”
- “We could show the percentage of the issues not scoped properly until the sprint started”

Annex B: User feedback to the productivity dashboard

Total number of respondents: 15 (11 female, 4 male).

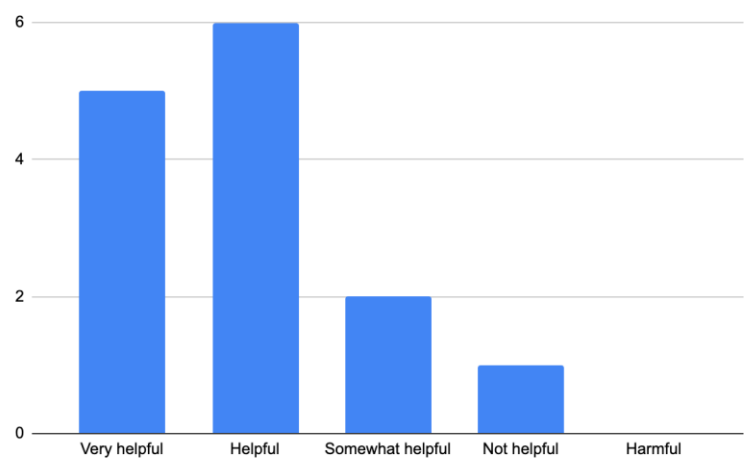
Age of respondents: 21-51.

Question 1: “How helpful do you feel the dashboard is for the team?”.



Picture 14 Question 1 answers distribution

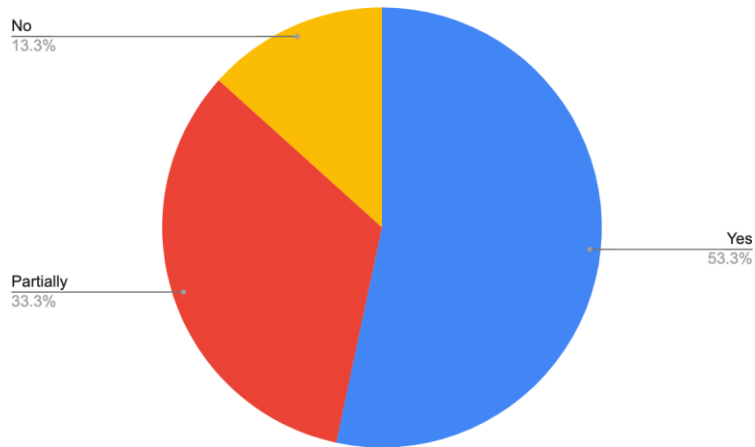
Question 2: “How helpful is the dashboard for you personally?”.



Picture 15 Question 2 answers distribution

Question 3: “Do the results align with your observations?”.

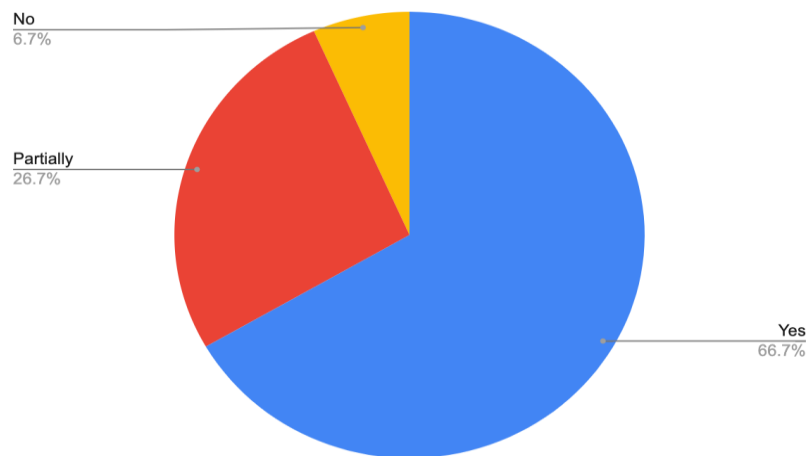
- Yes
- Partially
- No



Picture 16 Question 3 answers distribution

Question 4: “Do the results look trustworthy to you?”.

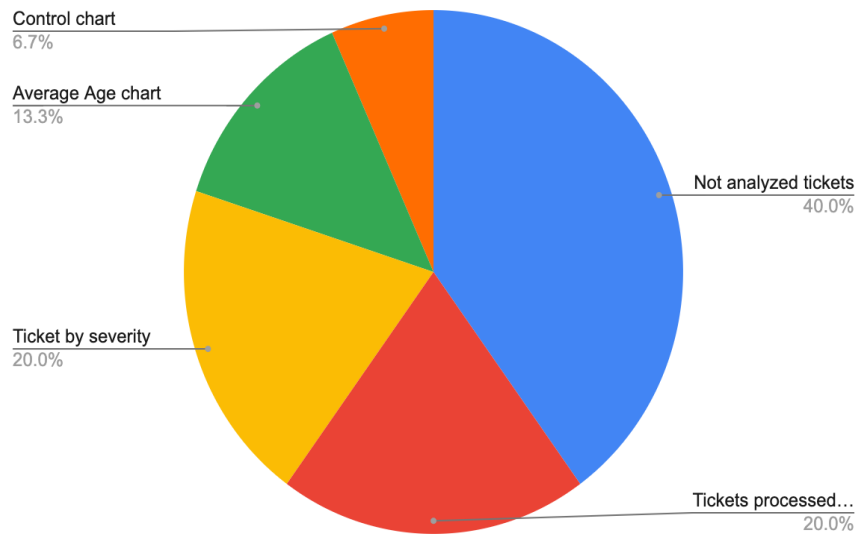
- Yes
- Partially
- No



Picture 17 Question 4 answers distribution

Question 5: “Which visualization do you find the most important?”.

- Not analyzed tickets
- Tickets processed this sprint
- Ticket by severity
- Average Age chart
- Control chart



Picture 18 Question 5 answers distribution